

VNU-HCM University of Information Technology

ASSIGNMENT 6

By Chu Cuong - 24520236

Email: cuongchu200@gmail.com
6-3-2025

Mục lục

Lời nói đầu.....	2
1. Longest Learning Sequence (LIS).....	3
2. Knapsack	6
3. Sum of Four Values	8
4. Meet In The Middle	10
5. Tìm đường đi trên đồ thị (BFS).....	12
6. Tìm đường đi trên đồ thị (DFS).....	15
7. ConnectedComponents.....	18
8. Đồ thị vô hướng – Đếm số đỉnh cô lập.....	20
9. Đồ thị vô hướng – Đếm số thành phần liên thông.....	22
10. Đồ thị vô hướng – Liệt kê các đỉnh có thể tới từ đỉnh S	24
11. Đồ thị vô hướng - Kiểm tra có đường đi từ đỉnh S tới E.....	26
12. Wildcard.....	28

My Github: [LINK](#)

Lời nói đầu

Bài tập tuần 6 có phần Basic với 12 bài tập được giao và đây là phần trình bày và cách mình làm những bài này:

[IT003.P21.CTTN.2] Assignment 6 (Basic) 

12 problems with a total score of 1200

#	PROBLEM	SCORE
1	Longest Increasing Sequence (LIS)	100
2	Knapsack	100
3	Sum of Four Values	100
4	Meet in the middle	100
5	Tìm đường đi trên đồ thị (BFS)	100
6	Tìm đường đi trên đồ thị (DFS)	100
7	khangtd.ConnectedComponents	100
8	Đồ thị vô hướng - Đếm số đỉnh cô lập	100
9	Đồ thị vô hướng - Đếm số thành phần liên thông	100
10	Đồ thị vô hướng - Liệt kê các đỉnh có thể tới từ đỉnh S.	100
11	Đồ thị vô hướng - Kiểm tra có đường đi từ đỉnh S tới đỉnh E	100
12	Wildcard	100

1. Longest Learning Sequence (LIS)

➤ Đề bài:

ROOKIE TRAINING SPRING 2023

LIS

Cho số nguyên dương n và dãy số A có n số nguyên $a_1, a_2, \dots, a_n$. Hãy tìm dãy con tăng dài nhất của A .

Định nghĩa dãy B được gọi là dãy con tăng độ dài K của dãy A nếu thỏa các điều kiện sau:

- $K \geq 1$.
- $1 \leq i_1 < i_2 < \dots < i_K \leq n$.
- $B_1 = a_{i_1}, B_2 = a_{i_2}, \dots, B_K = a_{i_K}$.
- $B_1 < B_2 < \dots < B_K$ nếu $K \geq 2$.

Dữ liệu vào: Gồm 2 dòng:

- Dòng đầu tiên chứa số nguyên dương n .
- Dòng thứ hai chứa n số nguyên $a_1, a_2, \dots, a_n$.

Kết quả: Gồm 2 dòng:

- Dòng đầu tiên in ra số nguyên dương K là độ dài dãy con tăng dài nhất của dãy A .
- Dòng thứ hai in ra K số nguyên $B_1, B_2, \dots, B_K$ là dãy con tăng dài nhất của dãy A . Nếu có nhiều dãy con tăng dài nhất thỏa mãn, in ra một dãy con tăng dài nhất thỏa mãn bất kì.

Ràng buộc: $1 \leq n \leq 5 \cdot 10^3$, $-10^9 \leq a_i \leq 10^9$.

Ví dụ:

Input	Output
5 2 4 3 1 5	3 2 4 5

➤ Ý tưởng:

- Đảo chiều mảng và đảo dấu $\rightarrow$ tìm dãy con giảm dài nhất từ trái sang phải.
- Sử dụng vector **dp** lưu các cặp (giá trị, chỉ số)
- Tìm vị trí chèn bằng tìm kiếm nhị phân (hàm lower_bound)
- Dùng prev[] để lưu vết, giúp truy vết lại LIS sau khi hoàn tất.
- Kết quả LIS cuối cùng được đảo lại do xây dựng từ cuối mảng.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e3 + 6;

vector<int> getLIS(int N, vector<int>& arr) {
    vector<pair<int, int>> dp;
    unordered_map<int, int> prv;

    // Process array in reverse order
    for (int ix = N - 1; ix >= 0; --ix) {
        int ve = -arr[ix];
        // Binary search to find insertion point
        auto it = lower_bound(dp.begin(), dp.end(), make_pair(ve, 0),
            [](const pair<int, int>& a, const pair<int, int>& b) {
                return a.first < b.first;
            });

        int tmp = -1; // Default previous index
        int i = distance(dp.begin(), it);

        if (i == dp.size()) {
            if (!dp.empty()) {
                tmp = dp.back().second;
            }
            dp.emplace_back(ve, ix);
        } else {
            if (i > 0) {
                tmp = dp[i-1].second;
            }
            dp[i] = {ve, ix};
        }
        prv[ix] = tmp;
    }

    // Reconstruct the LIS
    vector<int> res;
    int cur = dp.back().second;
    while (cur >= 0) {
        res.push_back(arr[cur]);
        cur = prv[cur];
    }

    return res;
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    vector<int> arr(n);
    for(int i = 0; i < n; i++)
        cin >> arr[i];

    vector<int> lis = getLIS(arr.size(), arr);
    cout << lis.size() << "\n";
    for (int num : lis) {
        cout << num << " ";
    }
    return 0;
}

```

2. Knapsack

➤ Đề bài:

ROOKIE TRAINING SPRING 2023

Knapsack

Cho n mặt hàng trong cửa hàng, mặt hàng thứ i nặng w_i , có giá trị v_i . Hiện bạn đang có túi đồ kích thước lớn nhưng sức chứa không quá W . Hãy tìm tổng giá trị lớn nhất có thể khi chọn một số mặt hàng trong cửa hàng mà không quá sức chứa của túi đồ.

Dữ liệu vào: Gồm $n + 1$ dòng:

- Dòng đầu tiên chứa hai số nguyên dương n, W .
- n dòng tiếp theo, dòng thứ i chứa hai số nguyên dương w_i, v_i .

Kết quả: In ra một dòng duy nhất là xâu con chung dài nhất. Nếu có nhiều xâu con chung dài nhất thoả mãn, in ra một xâu con chung dài nhất thoả mãn bất kì.

Ràng buộc: $n \leq 10^2, W \leq 10^5, 1 \leq w_i \leq 10^5, 1 \leq v_i \leq 10^9$.

Ví dụ:

Input	Output
3 10 4 5 5 6 6 7	12

➤ Ý tưởng:

- Duyệt từng mặt hàng i
 - Duyệt trọng lượng từ W xuống $wt[i - 1]$
 - Tại mỗi j , chọn giá trị lớn hơn giữa:
 - **Không lấy đồ i :** $dp[j]$
 - **Lấy đồ i :** $dp[j - wt[i - 1]] + val[i - 1]$

Cuối cùng, $dp[W]$ là kết quả cần tìm.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define ull unsigned long long

// Function to find the maximum profit
ull knapsack(int W, vector<int> &val, vector<int> &wt) {

    // Initializing dp vector
    vector<ull> dp(W + 1, 0);

    // Taking first i elements
    for (int i = 1; i <= wt.size(); i++) {

        // Starting from back, so that we also have data of
        // previous computation of i-1 items
        for (int j = W; j >= wt[i - 1]; j--) {
            dp[j] = max(dp[j], dp[j - wt[i - 1]] + (ull) val[i - 1]);
        }
    }
    return dp[W];
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n, W;
    cin >> n >> W;
    vector<int> wt(n), val(n);
    for(int i = 0; i < n; i++){
        cin >> wt[i] >> val[i];
    }

    cout << knapsack(W, val, wt) << "\n";
    return 0;
}
```


3. Sum of Four Values

➤ Đề bài:

UIT Algo Bootcamp Spring 2023

CSES - Sum of Four Values

Bạn được cho một mảng gồm N số nguyên và nhiệm vụ của bạn là tìm bốn giá trị (tại các vị trí phân biệt) có tổng là X .

Dữ liệu

- Dòng đầu tiên gồm số nguyên dương $N(N \leq 1000)$ và số nguyên $X(1 \leq X \leq 10^9)$.
- Dòng thứ hai chứa N số nguyên $a_i(1 \leq a_i \leq 10^9)$.

Kết quả

- In bốn số nguyên: vị trí của các giá trị. Nếu có một số lời giải, bạn có thể in bất kỳ lời giải nào trong số đó. Nếu không có lời giải nào, in **IMPOSSIBLE**.

Ví dụ

Sample Input	Sample Output
8 15 3 2 5 8 1 3 2 3	2 4 6 7

➤ Ý tưởng:

Ta sẽ tính tất cả các cặp tổng của 2 số trong N số rồi duyệt từng cặp và tìm cặp bù với X tương ứng trong các cặp còn lại mà thỏa mãn điều kiện 4 số này phân biệt.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define fi first
#define se second
#define ull unsigned long long

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int N, X;
    cin >> N >> X;
    vector<int> v(N + 1);
    for(int i = 1; i <= N; i++)
        cin >> v[i];

    map <ull, pair<int, int>> mp;

    for (int i = 1; i <= N; i++)
        for (int j = i + 1; j <= N; j++)
            mp[(ull) v[i] + v[j]] = make_pair(i, j);

    for (int i = 1; i <= N; i++) {
        for (int j = i + 1; j <= N; j++){
            if (mp.count(X - v[i] - v[j])) {
                pair<int, int> arr = mp[X - v[i] - v[j]];
                if (j < arr.fi) {
                    cout << i << ' ' << j << ' ' << arr.fi << ' ' << arr.se;
                    return 0;
                }
            }
        }
    }

    cout << "IMPOSSIBLE";
    return 0;
}
```

4. Meet In The Middle

➤ Đề bài:

UIT Algo Bootcamp Spring 2023

CSES - Meet In The Middle

Bạn được cho một mảng gồm N số. Có bao nhiêu cách chọn tập hợp con các số có tổng X .

Dữ liệu

- Dòng đầu tiên chứa hai số nguyên dương $N(N \leq 40)$ và $X(X \leq 10^9)$ lần lượt là kích thước mảng và tổng bắt buộc.
- Dòng tiếp theo chứa N số nguyên a_i ($1 \leq a_i \leq 10^9$).

Kết quả

- In ra số cách bạn có thể tạo ra tổng X .

Ví dụ

Sample Input	Sample Output
4 5 1 2 3 2	3

Giải thích

Có 3 cách chọn tập hợp để có tổng bằng 5:

- $a_1 + a_2 + a_4$
- $a_2 + a_3$
- $a_3 + a_4$

➤ Ý tưởng:

Dựa vào yêu cầu của đề thì ta sẽ chia làm 2 mảng và tính tổng tất cả các cặp trong 2 mảng này. Tiếp theo ta sắp xếp 2 mảng tổng, duyệt theo 1 mảng và tìm phần tử bù của nó trong mảng B mà tổng này bằng X . Đếm số lần và ta có được kết quả.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long
const int N = 40 + 2;

int t[N], n , x;
vector<ll> A, B;

void findsum(int i, int sum, int n, vector<ll>& v) {
    if (sum > x) return;

    if (i > n) v.push_back(sum);
    else {
        findsum(i + 1, (ll) sum, n, v);
        findsum(i + 1, (ll) sum + t[i], n, v);
    }
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> n >> x;
    for (int i = 1; i <= n; ++i) cin >> t[i];

    // Tất cả các cặp tổng của hai mảng
    findsum(1, 0, n / 2, A);
    findsum(n / 2 + 1, 0, n, B);

    // Sắp xếp mảng B
    sort(B.begin(), B.end());

    // Lặp qua mảng A và tìm kiếm nhị phân:
    // - Đếm số Lượng phần tử trong B có giá trị bằng x - A[i]
    ll cnt = 0;
    for (ll sum : A) {
        cnt += upper_bound(B.begin(), B.end(), x - sum)
              - lower_bound(B.begin(), B.end(), x - sum);
    }
    cout << cnt << '\n';
    return 0;
}
```

5. Tìm đường đi trên đồ thị (BFS)

➤ Đề bài:

Viết chương trình nhập vào 1 đồ thị có n đỉnh à e cạnh và m cặp đỉnh đầu, cuối cần tìm đường đi như ví dụ bên dưới
Chương trình xuất ra m chuỗi đường đi dùng BFS, nếu không có đường đi thì ghi ra *no_path*

Sample (Ví dụ):

Input	Output
6 13	U Q I
W K I U M Q	W M U K
W M	no_path
W Q	
I W	
I K	
I U	
I M	
U W	
U K	
U Q	
M U	
M Q	
Q I	
Q M	
3	
U I	
W K	
K M	

➤ Ý tưởng:

- Ta sẽ đánh số cho các điểm bằng hiệu của chúng với ký tự hoa đầu tiên.
- Tiếp theo, ta dùng thuật toán sử dụng CTDL hàng đợi và duyệt theo thứ tự ưu tiên chiều rộng.

➤ Script:

```
#include<bits/stdc++.h>
using namespace std;
const int N = 1e5 + 9;

vector<int> v[N];
int d[N], past[N], n, m;

void solve(char x, char y) {
    // Reset the distance and past arrays
    // to ensure they are ready for the new BFS search
    int s = x - 'A';
    int u = y - 'A';
    fill_n(d, 27, -1);
    fill_n(past, 27, -1);

    queue<int> q;
    q.push(s);
    d[s] = 0;
    while (!q.empty()) {
        int x = q.front();
        q.pop();
        for (auto y : v[x])
            if (d[y] == -1) {
                d[y] = d[x] + 1;
                past[y] = x;
                q.push(y);
            }
    }
    if (d[u] == -1) {
        cout << "no_path\n";
        return;
    }
    stack<char> res;
    while (u != -1) {
        res.push(char('A' + u));
        u = past[u];
    }
    while (!res.empty()) {
        cout << res.top() << " ";
        res.pop();
    }
    cout << "\n";
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cin >> n >> m;

    // pass the input of edges
    char x;
    for (int i = 1; i <= n; i++)
        cin >> x;

    // Read the edges and build the adjacency list
    for (int i = 1; i <= m; i++) {
        char x, y;
        cin >> x >> y;
        v[x - 'A'].push_back(y - 'A');
    }

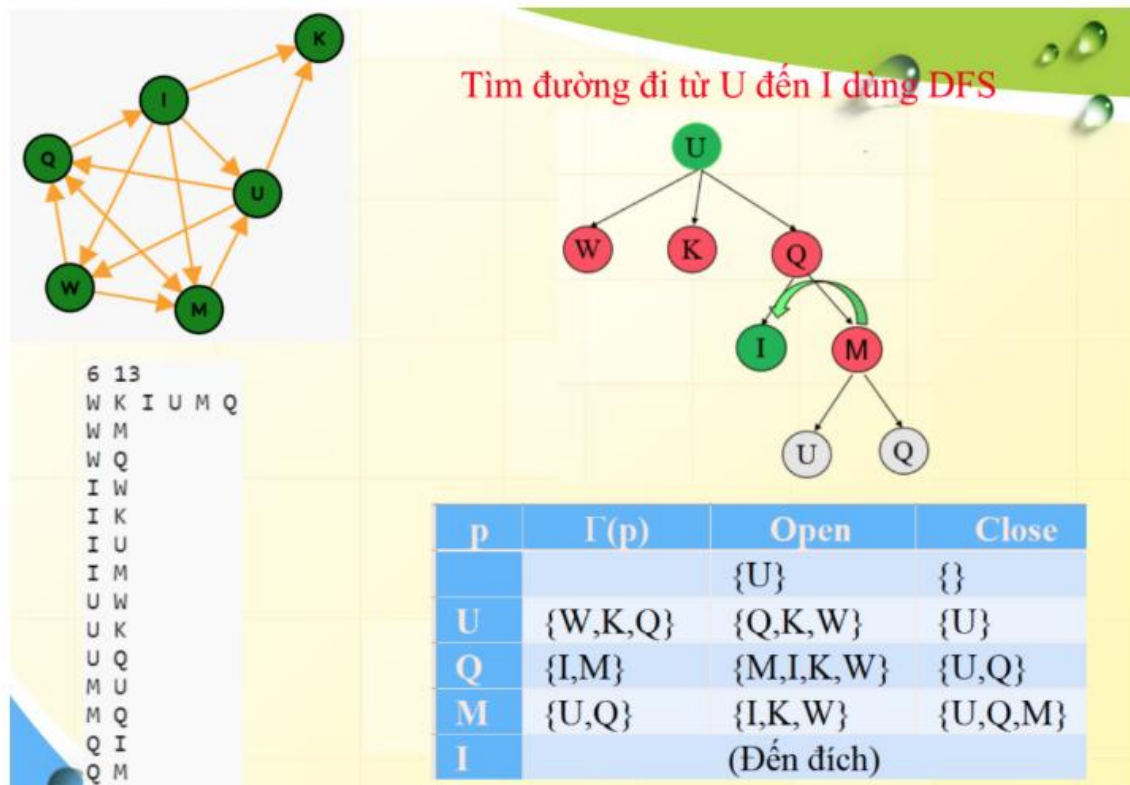
    cin >> m;
    while (m--) {
        char x, y;
        cin >> x >> y;
        solve(x, y);
    }
    return 0;
}

```

6. Tìm đường đi trên đồ thị (DFS)

➤ Đề bài:

Các bạn đã tìm hiểu lý thuyết về tìm đường đi trên đồ thị dùng DFS



Viết chương trình nhập vào 1 đồ thị có n đỉnh à e cạnh và m cặp đỉnh đầu, cuối cần tìm đường đi như ví dụ bên dưới
Chương trình xuất ra m chuỗi đường đi dùng DFS, nếu không có đường đi thì ghi ra *no_path*

➤ Ý tưởng:

- Ta sẽ đánh số cho các điểm bằng hiệu của chúng với ký tự hoa đầu tiên.
- Tiếp theo, ta dùng thuật toán sử dụng CTDL ngăn xếp và duyệt theo thứ tự ưu tiên chiều sâu.

➤ Script:

```
#include<bits/stdc++.h>
using namespace std;
const int N = 1e5 + 9;

vector<int> v[N];

int d[N], past[N], n, m;

void solve(char x, char y) {
    // Reset the distance and past arrays
    // to ensure they are ready for the new DFS search
    int s = x - 'A';
    int u = y - 'A';

    fill_n(d, 27, -1);
    fill_n(past, 27, -1);

    stack<int> q;
    q.push(s);
    d[s] = 0;
    while (!q.empty()) {
        int x = q.top();
        q.pop();
        for (auto y : v[x])
            if (d[y] == -1) {
                d[y] = d[x] + 1;
                past[y] = x;
                q.push(y);
            }
    }

    if (d[u] == -1) {
        cout << "no_path\n";
        return;
    }

    stack<char> res;
    while (u != -1) {
        res.push(char('A' + u));
        u = past[u];
    }
}
```

```

    while (!res.empty()) {
        cout << res.top() << " ";
        res.pop();
    }
    cout << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cin >> n >> m;

    // pass the input of edges
    char x;
    for (int i = 1; i <= n; i++)
        cin >> x;

    // Read the edges and build the adjacency list
    for (int i = 1; i <= m; i++) {
        char x, y;
        cin >> x >> y;
        v[x - 'A'].push_back(y - 'A');
    }

    cin >> m;
    while (m--) {
        char x, y;
        cin >> x >> y;
        solve(x, y);
    }
    return 0;
}

```

7. Connected Components

➤ Đề bài:

Thầy Sơn mới thiết kế một hệ thống N máy tính (các máy tính được đánh số từ 1 đến N) được nối lại thành một mạng bởi M kênh nối, mỗi kênh nối hai máy nào đó và cho phép ta truyền tin từ máy này sang máy kia. Nhưng không may rằng Thầy Sơn đã bị 1 hacker nào đó tấn công vào một máy bất kì trong N máy. Để khắc phục sự cố thì thầy Sơn cần liệt kê ra các máy đã bị tấn công. Mặc dù vậy nhưng mà hệ thống máy tính của thầy rất lớn nên không thể liệt kê bằng cách thông thường được. Em hãy viết chương trình giúp thầy xác định các máy bị tấn công? Biết rằng, nếu một máy bị tấn công thì các máy được kết nối trực tiếp cũng sẽ bị tấn công.



Dữ liệu:

- Dòng thứ nhất gồm số nguyên N, M ($1 \leq N, M \leq 10^6$). N là số máy tính và M là số kênh nối.
- M dòng tiếp theo, mỗi dòng gồm 2 số nguyên u và v ($1 \leq u \neq v \leq N$), cho biết máy u sẽ kết nối trực tiếp với máy v .
- Dòng cuối cùng chứa số nguyên s ($1 \leq s \leq N$), là số hiệu của máy bị tấn công.

Kết quả:

- Dòng đầu tiên in ra số lượng các máy bị tấn công.
- Dòng thứ hai in ra dãy số hiệu các máy tính bị tấn công theo thứ tự tăng dần.

➤ Ý tưởng:

Ta sẽ lưu trữ danh sách những máy nối với máy tương ứng bằng vector và rồi truy ngược với số hiệu của máy bị tấn công và in ra kết quả cần tìm.

➤ Script:

```
#include<bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int N, M;
    cin >> N >> M;
    vector<int> mt[N + 2];

    for(int i = 1; i <= M; i++){
        int temp1, temp2;
        cin >> temp1 >> temp2;
        mt[temp1].push_back(temp2);
        mt[temp2].push_back(temp1);
    }

    int target;
    cin >> target;

    set<int> res;
    int oldsize = res.size();
    res.insert(target);
    while(oldsize != res.size()){
        oldsize = res.size();
        for(auto i : res){
            for(auto x : mt[i])
                res.insert(x);
        }
    }

    cout << oldsize << "\n";
    for(auto i : res)
        cout << i << " ";
    return 0;
}
```

8. Đồ thị vô hướng – Đếm số đỉnh cô lập

➤ **Đề bài:**

Cho một đồ thị vô hướng G có tập đỉnh $V=\{0,1,2,...,m\}$ gồm m đỉnh và n cạnh. Hãy viết chương trình in ra số đỉnh không có cạnh nối trực tiếp với bất kỳ đỉnh nào khác trong G (số đỉnh cô lập).

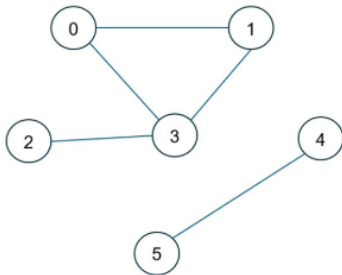
Đầu vào:

- Dòng đầu là hai số m, n cách nhau bằng khoảng trắng. Biết $1 \leq m \leq 1000, 0 \leq n \leq 3000$.

- n dòng kế tiếp, mỗi dòng 2 số nguyên u, v cho biết có cạnh giữa hai đỉnh u và v .

Đầu ra: một số nguyên là số đỉnh cô lập.

Ví dụ 1: Cho đồ thị như hình bên dưới



Input	Output
6 5	0
0 1	
0 3	
1 3	
2 3	
4 5	

➤ **Ý tưởng:**

Ta sẽ lưu trữ danh sách những máy nối với máy tương ứng bằng vector và rồi đếm số máy có danh sách này trống là được kết quả.

➤ Script:

```
#include<bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int m, n;
    cin >> m >> n;
    vector<int> g[m];

    for(int i = 0; i < n; i++){
        int temp1, temp2;
        cin >> temp1 >> temp2;
        g[temp1].push_back(temp2);
        g[temp2].push_back(temp1);
    }

    int cnt = 0;
    for(int i = 0; i < m; i++){
        if(g[i].empty()) cnt++;
    }

    cout << cnt;
    return 0;
}
```

9. Đồ thị vô hướng – Đếm số thành phần liên thông

➤ Đề thi:

Cho một đồ thị vô hướng G có tập đỉnh $V=\{0,1,2,\dots,m\}$ gồm m đỉnh và n cạnh. Hãy viết chương trình in ra số thành phần liên thông của G .

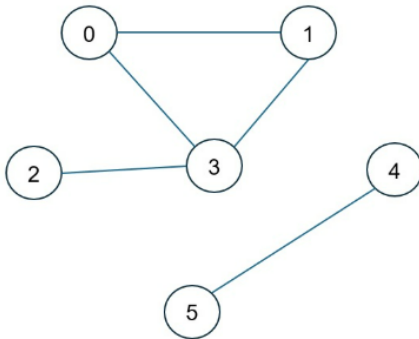
Đầu vào:

- Dòng đầu là hai số m, n cách nhau bằng khoảng trắng. Biết $1 \leq m \leq 1000, 0 \leq n \leq 3000$.

- n dòng kế tiếp, mỗi dòng 2 số nguyên u, v cho biết có cạnh giữa hai đỉnh u và v .

Đầu ra: một số nguyên là số thành phần liên thông của G .

Ví dụ 1: Cho đồ thị như hình bên dưới



Input	Output
6 5	2
0 1	
0 3	
1 3	
2 3	
4 5	

➤ Ý tưởng:

Ta sẽ lưu trữ danh sách những máy nối với máy tương ứng bằng vector và duyệt BFS để đếm số thành phần liên thông.

➤ Script:

```
#include<bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int m, n;
    cin >> m >> n;
    vector<int> g[m];
    bool visited[m];
    fill_n(visited, m, false);

    for(int i = 0; i < n; i++){
        int temp1, temp2;
        cin >> temp1 >> temp2;
        g[temp1].push_back(temp2);
        g[temp2].push_back(temp1);
    }
    int cnt = 0;
    for(int i = 0; i < m; i++){
        if(!visited[i]){
            cnt++;
            queue<int> q;
            q.push(i);
            visited[i] = true;

            while(!q.empty()){
                int top = q.front();
                q.pop();
                for(auto x : g[top]){
                    if(!visited[x]){
                        q.push(x);
                        visited[x] = true;
                    }
                }
            }
        }
    }
    cout << cnt;
    return 0;
}
```


10. Đồ thị vô hướng – Liệt kê các đỉnh có thể tới từ đỉnh S

➤ Đề bài:

Cho một đồ thị vô hướng G có tập đỉnh $V=\{0,1,2,...,m\}$ gồm m đỉnh và n cạnh. Hãy viết chương trình in ra tập D gồm tất cả các đỉnh có thể đi tới từ đỉnh 0. Biết các đỉnh trong tập D được sắp thứ tự tăng dần theo số hiệu đỉnh.

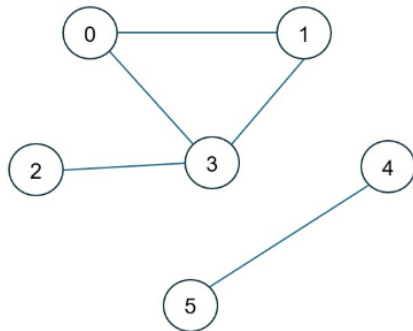
Đầu vào:

- Dòng đầu là hai số m, n cách nhau bằng khoảng trắng. Biết $1 \leq m \leq 1000, 0 \leq n \leq 3000$.

- n dòng kế tiếp, mỗi dòng 2 số nguyên u, v cho biết có cạnh giữa hai đỉnh u và v .

Đầu ra: Nếu tập D rỗng thì in ra "KHONG". Nếu $|D| > 0$ thì in ra tất cả đỉnh trong D , mỗi đỉnh cách nhau một khoảng trắng

Ví dụ 1: Cho đồ thị như hình bên dưới



Input	Output
6 5	1 2 3
0 1	
0 3	
1 3	
2 3	
4 5	

➤ Ý tưởng:

Ta sẽ lưu trữ danh sách những máy nối với máy tương ứng bằng vector và rồi truy ngược với đỉnh 0 và in ra kết quả cần tìm.

➤ Script:

```
#include<bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int m, n;
    cin >> m >> n;
    vector<int> g[m];

    for(int i = 0; i < n; i++){
        int tmp1, tmp2;
        cin >> tmp1 >> tmp2;
        g[tmp1].push_back(tmp2);
        g[tmp2].push_back(tmp1);
    }

    if(g[0].empty()) cout << "KHONG";
    else{
        set<int> s;
        for(auto i : g[0])
            s.insert(i);
        int oldsize = -1;
        while(oldsize != s.size()){
            oldsize = s.size();
            for(auto i : s){
                for(auto x : g[i])
                    s.insert(x);
            }
        }
        s.erase(0);
        for(auto i : s)
            cout << i << " ";
    }
    return 0;
}
```

11. Đồ thị vô hướng - Kiểm tra có đường đi từ đỉnh S tới E

➤ Đề bài:

Cho một đồ thị vô hướng G có tập đỉnh $V = \{0, 1, 2, \dots, m\}$ gồm m đỉnh và n cạnh. Hãy viết chương trình cho biết có đường đi từ đỉnh 0 đến các đỉnh còn lại của đồ thị G hay không.

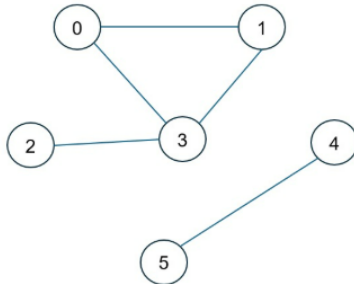
Đầu vào:

- Dòng đầu là hai số m, n cách nhau bằng khoảng trắng. Biết $1 \leq m \leq 1000, 0 \leq n \leq 3000$.

- n dòng kế tiếp, mỗi dòng 2 số nguyên u, v cho biết có cạnh giữa hai đỉnh u và v .

Đầu ra: gồm $m-1$ dòng. Dòng thứ i cho biết giữa đỉnh 0 đến đỉnh i có đường đi hay không. Nếu có thì ghi kết quả "CO", nếu không thì ghi kết quả "KHONG".

Ví dụ: Cho đồ thị như hình bên dưới



Input	Output
6 5	CO (giải thích: có đường đi từ đỉnh 0 đến đỉnh 1)
0 1	CO (giải thích: có đường đi từ đỉnh 0 đến đỉnh 2)
0 3	CO (giải thích: có đường đi từ đỉnh 0 đến đỉnh 3)
1 3	KHONG (giải thích: không có đường đi từ đỉnh 0 đến đỉnh 4)
2 3	KHONG (giải thích: không có đường đi từ đỉnh 0 đến đỉnh 5)
4 5	

➤ Ý tưởng:

Sử dụng ý tưởng truy vết như những bài trên.

➤ Script:

```
#include<bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int m, n;
    cin >> m >> n;
    vector<int> g[m];
    for(int i = 0; i < n; i++){
        int tmp1, tmp2;
        cin >> tmp1 >> tmp2;
        g[tmp1].push_back(tmp2);
        g[tmp2].push_back(tmp1);
    }

    if(g[0].empty())
        for(int i = 1; i < m; i++)
            cout << "KHONG" << "\n";
    else{
        set<int> s;
        for(auto i : g[0])
            s.insert(i);
        int oldsize = -1;
        while(oldsize != s.size()){
            oldsize = s.size();
            for(auto i : s){
                for(auto x : g[i])
                    s.insert(x);
            }
        }
        for(int i = 1; i < m; i++){
            auto it = s.find(i);
            if (it != s.end())
                cout << "CO" << "\n";
            else
                cout << "KHONG" << "\n";
        }
    }
    return 0;
}
```

12. Wildcard

➤ Đề bài:

Wildcard

Trang Catbook là một mạng xã hội dành cho những người nuôi mèo. Trên trang, người ta có thể trao đổi tin nhắn, chia sẻ với nhau những thông tin bổ ích về việc nuôi mèo, cũng như những khoảnh khắc thú vị bên những con mèo của mình.

Trang web đã rất tuyệt vời, tuy nhiên tính năng khi người ta muốn tìm kiếm lại những thông tin vẫn chưa thực sự tốt lắm. Chính vì vậy nên các lập trình viên trong đội ngũ được giao nhiệm vụ phát triển hệ thống tìm kiếm. Bài toán tìm kiếm ở đây khá phức tạp nên được chia ra thành nhiều bài toán nhỏ, một trong số bài toán đó là:

- ❑ Bạn được cho **N** mẫu tin, mỗi mẫu tin chỉ chứa các ký tự (a..z, A..Z, 0..9 và dấu khoảng trắng '\32'). Ngoài ra bạn cũng được cho **M** từ khóa, với mỗi từ khóa thì tìm xem có bao nhiêu tin nhắn có thể khớp với từ khóa đó?

Điểm đặc biệt ở đây là mỗi từ khóa sẽ có tối đa thêm 1 dấu '*', với ý nghĩa là có thể thay thế '*' thành một chuỗi ký tự bất kỳ, có thể là chuỗi rỗng. Ví dụ từ khóa "ex*" có thể khớp với "ex", "exchange", "excel" nhưng không khớp với "empty"; từ khóa "**ant" thì có thể khớp với "ant", "important" nhưng không khớp với "hint".

Các lập trình viên đang không biết nên giải quyết vấn đề này như thế nào cho hiệu quả, nên xin nhờ sự giúp đỡ của các bạn. Hãy giúp đỡ các anh lập trình viên của chúng ta nhé!

Input

- Dòng đầu tiên chứa 2 số nguyên **N** và **M** tương ứng với số mẫu tin và số từ khóa
- **N** dòng tiếp theo, mỗi dòng chứa một mẫu tin
- **M** dòng tiếp theo, mỗi dòng chứa một từ khóa

Output

- Gồm **M** dòng, dòng thứ *i* chứa một số thể hiện số lượng mẫu tin khớp với từ khóa thứ *i*

➤ Ý tưởng:

Ta sẽ dùng thuật toán Rabin Karp – Double Hash để giảm khả năng trùng hash và tìm xâu kí tự trùng nhau trong thời gian ngắn.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define fi first
#define se second
using ll = long long;
using H = pair<ll, ll>;

const ll M1 = 998244353, M2 = 1000000007;
const ll B1 = 307, B2 = 311;
vector<H> h, p;

struct Hash{
    void build(string s) {
        int n = s.size();
        h.assign(n + 1, {0, 0});
        p.assign(n + 1, {1, 1});
        for (int i = 1; i <= n; ++i){
            p[i] = {p[i - 1].fi * B1 % M1,
                    p[i - 1].se * B2 % M2};
            h[i] = {(h[i - 1].fi * B1 + s[i - 1] + 3) % M1,
                    (h[i - 1].se * B2 + s[i - 1] + 3) % M2};
        }
    }

    H get(int l, int r){
        if (l > r)
            return {0, 0};
        ll x = (h[r].fi - h[l - 1].fi * p[r - l + 1].fi + M1 * M1) % M1;
        ll y = (h[r].se - h[l - 1].se * p[r - l + 1].se + M2 * M2) % M2;
        return {x, y};
    }
} hs;

map<H, int> pre, suf, full;
map<pair<H, H>, int> mp_ps; // (prefix-hash , suffix-hash)

void add_string(string s){
    hs.build(s);
    int n = s.size();

    full[hs.get(1, n)]++; // khớp hoàn toàn
    for (int i = 1; i <= n; ++i){
        pre[hs.get(1, i)]++;
```

```

        suf[hs.get(i, n)]++;
        for (int j = i + 1; j <= n; ++j) // vị trí '*' giữa i và j-1
            mp_ps[{hs.get(1, i), hs.get(j, n)}]++;
    }
}

int query(string s, int n){
    if (s.empty()) return 0;
    hs.build(s);
    int m = s.size();
    if (s == "*")
        return n;
    if (s.find('*') == -1)
        return full[hs.get(1, m)];
    if (s[0] == '*')
        return suf[hs.get(2, m)];
    if (s[m - 1] == '*')
        return pre[hs.get(1, m - 1)];

    int k = s.find('*');
    return mp_ps[{hs.get(1, k), hs.get(k + 2, m)}];
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n, q;
    cin >> n >> q;
    string s;
    getline(cin, s);

    for (int i = 1; i <= n; i++) {
        string s;
        getline(cin, s);
        add_string(s);
    }

    while (q--){
        string s;
        getline(cin, s);
        cout << query(s, n) << '\n';
    }
    return 0;
}

```